PUNTAL AGRO

Manual técnico y operativo Publicación autónoma de Tableros de Decisión

SFTP restringido · Staging · READY · Cron · Backup · Docker · Rollback

Edición: 16 de octubre de 2026. Base documental: implementación y pruebas registradas en la conversación del proyecto Puntal Agro (septiembre de 2026).

Propósito: ayuda memoria para operación, soporte, transferencia de conocimiento y futuras modificaciones. No es una auditoría en vivo del VPS: los comandos de consulta permiten verificar el estado actual.

Estado verificado históricamente	Alcance
Operativo y probado	SFTP con usuario aislado; publicación manual y automática; backups; rollback del tablero Comercial Agrícola.
Pendiente de confirmar en vivo	Estado actual de archivos y permisos, ejecución horaria, y adopción/pruebas del skill ampliado por el cliente.

1. Resumen ejecutivo

Se implementó un canal de publicación separado de la aplicación de tres capas. El cliente genera HTMLs de cuatro Tableros de Decisión y los deposita mediante SFTP en un área de staging. Un script local del VPS, ejecutado por cron, detecta un archivo marcador READY, valida el contenido, guarda una copia de la versión productiva y sincroniza los archivos a un directorio persistente del host. Ese directorio está montado dentro del contenedor Node/Express; por lo tanto, los archivos se reflejan sin reconstruir la imagen ni reiniciar el contenedor.

Decisión de arquitectura central: el usuario SFTP nunca escribe directamente en producción ni dentro del contenedor. La publicación la realiza un proceso local administrado por el servidor.

2. Diagrama de arquitectura y flujo

```
[Cliente + Claude / skill de publicación]
             | SFTP (SSH, puerto 722)
             v
[Chroot SFTP: /srv/puntal_sftp]
             |
             v
[/srv/puntal_sftp/uploads/<tablero>/]
   index.html + recursos + READY
             | cron: 0 * * * *
             v
[/usr/local/bin/publicar-tableros-puntal.sh]
   1) Detectar READY
   2) Validar estructura / extensiones
   3) Backup de producción actual
   4) rsync al frontend del host
   5) Eliminar READY y registrar log
             |
             v
[/var/www/desa/dg/puntal3c/frontend/tableros-decision/<tablero>/]
             | bind mount: ./frontend:/usr/src/frontend
             v
[Docker: puntal3c_express_api]
   /usr/src/frontend/tableros-decision/<tablero>/
             |
             v
[Navegador: http://ap2.puntalagro.com:3003/...]
```

3. Alcance y separación de responsabilidades

Componente	Responsabilidad
Claude / skill	Identificar tablero, preparar carpeta, transferir por SFTP, verificar y crear READY al final.
Usuario Linux SFTP	Acceso limitado a staging; sin shell ni acceso administrativo a producción.
Script del VPS	Validar, respaldar y copiar a producción.
Cron	Invocar periódicamente el script; no decide contenido ni cambia permisos.
Docker / Express	Servir los archivos visibles mediante bind mount.
Administrador	Monitoreo, diagnóstico, restauración, cambios de frecuencia y mantenimiento.

4. Los cuatro tableros y sus destinos

Tablero	Archivo fuente	Carpeta de staging
Comercial Agrícola	tablero_agro.html	/uploads/comercial-agricola/
Evolución de Variables	tablero_evolucion.html	/uploads/evolucion-variables/
Relaciones Ganaderas	tablero_hacienda.html	/uploads/relaciones-ganaderas/
Tarifas de Labores y Fletes	tablero_labores.html	/uploads/labores-fletes/

En todos los casos, el archivo fuente se transfiere con el nombre index.html. Cada carpeta también debe recibir READY únicamente después de completar la carga. Las rutas de la tabla son las que ve el usuario dentro del SFTP, no rutas absolutas del sistema Linux.

5. Rutas reales del VPS y Docker

Uso	Ruta
Chroot del usuario SFTP	/srv/puntal_sftp
Staging físico	/srv/puntal_sftp/uploads
Script	/usr/local/bin/publicar-tableros-puntal.sh
Cron	/etc/cron.d/puntal-tableros
Backups de tableros	/srv/puntal-tableros-backup
Log de publicaciones	/var/log/puntal-tableros-deploy.log
Frontend en el host	/var/www/desa/dg/puntal3c/frontend/tableros-decision
Frontend en el contenedor	/usr/src/frontend/tableros-decision

Detalle histórico importante: al principio se utilizó por error la ruta `/srv/puntal-sftp` (guion). La ruta real confirmada con `ls -li /srv` fue /srv/puntal_sftp (guion bajo). Se corrigió la variable STAGING del script y la publicación empezó a funcionar.

6. Cuenta Linux y aislamiento SFTP

Cuenta configurada: puntal_tableros_upload. El SSH del VPS escuchaba en el puerto 722. Se verificó la conexión desde WinSCP y que el usuario viera solamente `/uploads`. La configuración se agregó en `/etc/ssh/sshd_config`:

```
Match User puntal_tableros_upload
    ChrootDirectory /srv/puntal_sftp
    ForceCommand internal-sftp
    AllowTcpForwarding no
    X11Forwarding no
    PermitTunnel no
```

La raíz del chroot debe pertenecer a root y no ser escribible por el usuario. La subcarpeta uploads sí debe permitirle escribir. Las cuatro carpetas fueron creadas bajo uploads y se recomendó permiso 750 para limitar lectura por terceros. Se validó la sintaxis con `sudo sshd -t` y se aplicó con `sudo systemctl reload ssh`, manteniendo una sesión administrativa abierta.

```
# Consultas de estado; no cambian la configuración
id puntal_tableros_upload
sudo ls -ld /srv/puntal_sftp /srv/puntal_sftp/uploads
sudo ls -ld /srv/puntal_sftp/uploads/*
sudo sshd -t
sudo systemctl status ssh
```

7. Contrato de publicación: READY

READY es un archivo vacío que actúa como señal de fin de transferencia. El script no publica si no lo encuentra. Tras una publicación correcta, el script lo elimina. Esto permite que el cron quede activo permanentemente sin volver a publicar versiones antiguas.

```
/uploads/comercial-agricola/
    index.html
    css/        (opcional)
    js/         (opcional)
    images/     (opcional)
    READY       (creado como última operación)
```

El orden importa: el cliente debe subir primero todos los recursos y verificar su presencia. Si ocurre un error, NO crear READY. Si quedó un READY antiguo antes de iniciar una nueva transferencia, debe retirarse antes de reemplazar archivos.

8. Funcionamiento del script de publicación

El script creado y probado fue `/usr/local/bin/publicar-tableros-puntal.sh`, propiedad de root y con permisos restrictivos. Utiliza las variables STAGING, PRODUCCION, BACKUP y LOG y una lista cerrada de cuatro nombres de tablero.

Comportamiento registrado: para cada tablero comprueba READY; valida la existencia de index.html, rechaza enlaces simbólicos y extensiones no autorizadas; crea una carpeta de backup con timestamp y copia el contenido productivo existente; luego ejecuta rsync con --delete para sustituir la versión publicada, excluyendo READY y ERROR; elimina READY y registra el éxito.

```
STAGING="/srv/puntal_sftp/uploads"
PRODUCCION="/var/www/desa/dg/puntal3c/frontend/tableros-decision"
BACKUP="/srv/puntal-tableros-backup"
LOG="/var/log/puntal-tableros-deploy.log"

# Fragmento esencial del proceso, no el script completo:
rsync -a --delete --exclude="READY" --exclude="ERROR"   "$origen/" "$destino/"
rm -f "$origen/READY"
```

Precisión: el script actual se documenta según la versión compartida en el chat y las pruebas realizadas. Si fue modificado luego en el VPS, inspeccionar el archivo real antes de tomar este comportamiento como definitivo.

9. Cron: ejecución horaria

Se inició con una ejecución cada minuto para probar, luego cada cinco minutos y finalmente se cambió, por pedido del administrador, a una vez por hora, al minuto cero.

```
# /etc/cron.d/puntal-tableros
0 * * * * root flock -n /var/run/puntal-tableros.lock /usr/local/bin/publicar-tableros-puntal.sh
```

El uso de flock evita ejecuciones simultáneas del mismo job. La modificación de /etc/cron.d normalmente es detectada por cron sin reiniciar el servicio. Si no hay READY, el script no publica ese tablero.

```
cat /etc/cron.d/puntal-tableros
sudo systemctl status cron
sudo journalctl -u cron --since "1 hour ago" | tail -30
sudo tail -30 /var/log/puntal-tableros-deploy.log
```

10. Relación con Docker y Express

En el compose utilizado, el servicio de API se llama puntal3c_api y su contenedor puntal3c_express_api. El frontend del host se monta en el contenedor mediante:

```
volumes:
  - ./frontend:/usr/src/frontend
```

Por eso, al sincronizar a `.../frontend/tableros-decision/`, los archivos aparecen automáticamente en `/usr/src/frontend/tableros-decision/`. No se necesita docker-compose up, build, down ni restart para cambios de HTML/CSS/JS de estos tableros.

Distinto es modificar backend/`server.js`: al no estar montado como volumen en la configuración documentada, se reconstruye la imagen de la API con `docker-compose up -d --build puntal3c_api`. Esa operación no debe formar parte del skill de publicación.

```
# Verificación del archivo desde el contenedor
docker exec puntal3c_express_api   ls -la /usr/src/frontend/tableros-decision/comercial-agricola/

# Verificar contenedores
docker ps | grep -E 'puntal3c_express_api|puntal3c_postgres_db'
```

11. Procedimiento operativo: publicar un tablero

• Preparar: identificar uno de los cuatro tableros y reunir index.html y todos sus recursos locales en su propia carpeta.

• Transferir: conectarse mediante SFTP, puerto 722, usuario puntal_tableros_upload, al directorio /uploads/<tablero>/.

• Seguridad del marcador: retirar READY antiguo antes de empezar; subir los archivos; verificar transferencia y referencias relativas.

• Habilitar: crear READY como última operación.

• Esperar: el cron horario detectará READY; no hace falta ejecutar nada en Docker.

• Verificar: revisar log, ausencia de READY, carpeta productiva y navegador (Ctrl+F5 si hay caché).

```
# Consultas para Comercial Agrícola
sudo ls -la /srv/puntal_sftp/uploads/comercial-agricola
sudo tail -30 /var/log/puntal-tableros-deploy.log
ls -la /var/www/desa/dg/puntal3c/frontend/tableros-decision/comercial-agricola
```

La ruta web probada históricamente para Comercial Agrícola fue http://ap2.puntalagro.com:3003/tableros-decision/comercial-agricola/. Las rutas equivalentes de los otros tres tableros siguen el mismo patrón, pero conviene verificar su exposición actual en Express.

12. Backups: ubicación, contenido y retención

Antes de sobrescribir la versión productiva de un tablero, el script guarda una copia completa de su carpeta en `/srv/puntal-tableros-backup/<tablero>/AAAA-MM-DD_HHMMSS/`. En las pruebas de Comercial Agrícola se confirmaron varias carpetas de backup y su contenido.

```
/srv/puntal-tableros-backup/
    comercial-agricola/
        2026-09-03_154000/
            index.html
        2026-09-03_162300/
            index.html
    evolucion-variables/
    relaciones-ganaderas/
    labores-fletes/
```

Los timestamps anteriores son ilustrativos. El backup de tableros es distinto del dump de PostgreSQL. No respalda la base de datos, el backend, ni todo el proyecto. La retención automática de 60 días se discutió como posible mejora, pero no consta que se haya activado. Revisar espacio disponible periódicamente.

```
sudo ls -lt /srv/puntal-tableros-backup/comercial-agricola/
sudo du -sh /srv/puntal-tableros-backup
df -h /srv
```

13. Rollback: restauración de versión anterior

El rollback se probó exitosamente. No es necesario detener cron si no existe READY en staging. El backup elegido se sincroniza hacia la carpeta productiva del host y Docker ve el cambio inmediatamente.

```
# 1. Confirmar que no haya una publicación pendiente
sudo ls -la /srv/puntal_sftp/uploads/comercial-agricola/

# 2. Identificar el backup adecuado
sudo ls -lt /srv/puntal-tableros-backup/comercial-agricola/

# 3. Restaurar (sustituir NOMBRE_BACKUP)
sudo rsync -a --delete   /srv/puntal-tableros-backup/comercial-agricola/NOMBRE_BACKUP/   /var/www/
desa/dg/puntal3c/frontend/tableros-decision/comercial-agricola/
```

Precaución: rsync --delete elimina del destino los archivos ausentes en el backup elegido. Comprobar tres veces tablero y nombre de backup. Si hay READY pendiente, coordinar o bloquear esa publicación antes de restaurar para evitar que el cron vuelva a reemplazar el rollback.

Tras restaurar, abrir el tablero y forzar recarga de caché. No es necesario reiniciar el contenedor.

14. Skill de Claude: contrato funcional

Se entregaron instrucciones para que el cliente ampliara su skill de Comercial Agrícola a los cuatro tableros. El skill se ejecuta en el entorno Claude del cliente, al que el administrador no tiene acceso directo. El cliente debe instalar/configurar una capacidad real de SFTP; las instrucciones del skill por sí solas no crean conectividad.

Solicitud del cliente	Archivo a enviar	Destino SFTP
Publicar Comercial Agrícola	tablero_agro.html	/uploads/comercial-agricola/index.html
Publicar Evolución de Variables	tablero_evolucion.html	/uploads/evolucion-variables/index.html

Solicitud del cliente	Archivo a enviar	Destino SFTP
Publicar Relaciones Ganaderas	tablero_hacienda.html	/uploads/relaciones-ganaderas/index.html
Publicar Tarifas de Labores y Fletes	tablero_labores.html	/uploads/labores-fletes/index.html

El skill debe utilizar únicamente destinos de la tabla; incluir CSS, JS, JSON e imágenes locales cuando existan; comprobar index.html; no generar READY en caso de error; no tocar backend, PostgreSQL, Docker ni producción; y comunicar únicamente que la carga quedó lista para publicación automática, no que ya está publicada.

Credenciales: no incluir contraseñas en repositorios, prompts públicos ni el texto del skill si el entorno permite usar secretos o credenciales protegidas. Preferir autenticación con clave SSH dedicada y protegida, cuando esté disponible.

15. Guía de diagnóstico y resolución de problemas

Síntoma	Comprobación / acción
El cron no publica	Verificar READY exacto, cron activo, log y ruta STAGING con guion bajo.
No se genera backup	Revisar si el script entró en la rama READY y si había versión productiva. Consultar log.
Se subió HTML pero no cambia la web	Comprobar ausencia/presencia de READY, log, archivo en host, bind mount y caché del navegador.
WinSCP no conecta	Revisar host, puerto 722, usuario, credenciales, servicio SSH y configuración Match User.
Error de permisos	Verificar dueño/permisos de chroot, uploads, staging, producción, backups y script.
Publicación rechazada	Comprobar index.html, enlaces simbólicos, extensiones permitidas y archivo ERROR.
La API no sirve la ruta	Comprobar montaje y configuración de rutas estáticas en server.js.
Un rollback no se mantiene	Comprobar si quedó READY en staging y si el cron publicó otra versión después.
El comando sudo avisa unable to resolve host	Revisar correspondencia del hostname puntalagro en /etc/hostname y /etc/hosts; problema separado del cron.

16. Comandos de consulta rápida

```
# Staging
sudo ls -la /srv/puntal_sftp/uploads/*/

# Cron
cat /etc/cron.d/puntal-tableros
sudo systemctl status cron

# Log de publicaciones
sudo tail -50 /var/log/puntal-tableros-deploy.log

# Backups
sudo ls -la /srv/puntal-tableros-backup/

# Script real
sudo sed -n '1,240p' /usr/local/bin/publicar-tableros-puntal.sh

# Archivos productivos
ls -la /var/www/desa/dg/puntal3c/frontend/tableros-decision/

# Contenedor
sudo docker ps | grep puntal3c_express_api
```

17. Limitaciones del diseño actual y mejoras recomendadas

El sistema funciona y fue probado, pero el script compartido no implementa una publicación de carpeta completamente atómica. rsync modifica el directorio productivo mientras copia; un visitante podría acceder durante esa ventana. Asimismo, el cliente puede reemplazar archivos de staging antes de colocar READY y no existe una reserva exclusiva de staging por release.

• Prioridad alta: preparar cada versión en una carpeta de release nueva y hacer cambio atómico de puntero o directorio después de validarla, en lugar de sincronizar directamente al destino público.

• Prioridad alta: proteger la publicación frente a fallos a mitad de rsync; conservar staging/READY y registrar estado de error recuperable; considerar validación de checksums y límites de tamaño/cantidad.

• Prioridad media: validar que no haya archivos especiales, nombres peligrosos o rutas fuera del árbol; restringir tipos de archivos según necesidades reales; limitar contenido activo potencialmente inseguro.

• Prioridad media: agregar rotación de logs y política explícita de retención de backups, con monitoreo de espacio.

• Prioridad media: agregar manifiesto de versión (por ejemplo deploy.json) con fecha, tablero, identificador y checksum, validado por el script. Se propuso, pero no consta su implementación.

• Prioridad media: evitar usar la cuenta root para toda la lógica si se puede aislar el proceso con permisos mínimos, sin perder capacidad de escribir en destino y backup.

• Operación: establecer procedimiento de revocación/rotación de credenciales y registro de quién publica; revisar el skill en el entorno del cliente.

18. Checklist de operación y recuperación

Antes de publicar: archivo correcto; carpeta correcta; recursos completos; READY anterior retirado; transferencia SFTP terminada.

Después de publicar: log con éxito; READY eliminado; backup presente si había versión anterior; archivos productivos correctos; página accesible.

Antes de rollback: identificar backup y tablero; comprobar READY; confirmar que la restauración no pisa cambios de otra persona; guardar evidencia si hace falta.

Después de rollback: confirmar versión restaurada y navegación; revisar log/cron para descartar nueva publicación automática.

19. Cronología de la implementación

Etapa	Resultado registrado
Usuario y chroot	Se creó puntal_tableros_upload; WinSCP mostró /uploads.
Carpetas staging y producción	Se crearon cuatro directorios para los tableros.
Script y backup	Se detectó y corrigió la diferencia puntal-sftp vs puntal_sftp.
Pruebas manuales	Se publicaron versiones y se comprobaron dos backups.
Rollback	Se restauró una versión anterior y se verificó en navegador.
Automatización	Se probó cron cada 5 minutos y luego se configuró cada hora.
Skill	Se redactaron instrucciones para publicar los cuatro tableros desde Claude del cliente.

20. Conclusión

La solución separa la capacidad de entrega del cliente de la administración de la aplicación. SFTP limita el alcance del usuario; READY explicita cuándo una versión está lista; el cron y el script controlan el paso a producción; el backup permite volver a una versión anterior; y el bind mount de Docker evita despliegues innecesarios de la API. Para cambios frecuentes de HTML autónomos, es una solución simple, mantenible y de bajo acoplamiento con el sistema de tres capas.

Nota de vigencia: este documento recoge la implementación que consta en el historial del proyecto. Antes de intervenir un VPS en producción, confirmar las rutas y el contenido real del script, el cron y docker-compose.